

Section 11.7 of 10: LangChain Ecosystem — LangGraph, LangSmith & Production Patterns

Executive Overview

LangChain provides composable abstractions for AI application development. LangGraph enables stateful, durable agent workflows using a graph/state-machine model; LangSmith adds observability, evaluation, and debugging. Together they form a production-grade stack for agentic systems. This section covers LCEL composition, LangGraph state management, and practical deployment patterns.

11.7.1 LangChain Expression Language (LCEL)

Declarative Composition with Pipe Operator

LCEL treats every component as a `Runnable` — a uniform interface with `invoke`, `stream`, `batch`, and `async` variants. Components are chained with `|`.

```
from langchain_core.runnables import RunnableSequence
from langchain_core.output_parsers import StrOutputParser

# Each | creates a Runnable pipeline
chain = (
    prompt_template      # Runnable: formats user input into prompt
    | model               # Runnable: calls LLM, returns AIMessage
    | StrOutputParser()   # Runnable: extracts text from AIMessage
    | database_lookup     # Runnable: uses extracted text as DB key
)

# Invocation modes — same chain, different behavior:
result = chain.invoke({"question": "What is AI?"})      # Sync
result = await chain.ainvoke({"question": "..."})      # Async
results = chain.batch([{"question": "Q1"}, {"question": "Q2"}]) # Parallel batch

# Streaming — automatic for all chains
async for chunk in chain.astream({"question": "..."}):
    print(chunk, end="", flush=True)
```

LCEL Benefits vs. Manual Chains

Feature	Manual Implementation	LCEL
Composition	Custom wrapper code per step	<code> </code> pipe operator
Streaming	Implement per step	Automatic propagation
Batching	Manual parallelism	Automatic
Async	Convert each step	Automatic async variants
Retries	Custom retry logic	<code>.with_retry(stop_after_attempt=3)</code>
Fallbacks	Custom try/except	<code>.with_fallbacks([backup_model])</code>

Parallelism in LCEL

```
from langchain_core.runnables import RunnableParallel

# Run two chains simultaneously, merge results
parallel_chain = RunnableParallel(
    summary=summarize_chain,    # LLM call 1
    keywords=extract_chain      # LLM call 2 (runs in parallel)
)

result = parallel_chain.invoke({"text": document})
# result = {"summary": "...", "keywords": [...]}
# Total latency = max(summarize_time, extract_time) rather than sum
```

11.7.2 LangGraph — Stateful Agent Runtime

Graph Model

LangGraph represents agent workflows as directed graphs where nodes are processing functions and edges define control flow. This enables cycles (for loops), conditionals, and checkpointing.

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class AgentState(TypedDict):
    messages: List[dict]
    tool_calls: List[dict]
    iteration_count: int
    final_answer: str | None
```

```

def reason(state: AgentState) -> AgentState:
    # LLM call with tools
    response = model_with_tools.invoke(state["messages"])
    return {
        "messages": state["messages"] + [response],
        "iteration_count": state["iteration_count"] + 1
    }

def act(state: AgentState) -> AgentState:
    # Execute tool calls from last message
    results = execute_tools(state["messages"][-1].tool_calls)
    return {"messages": state["messages"] + results}

def should_continue(state: AgentState) -> str:
    last_message = state["messages"][-1]
    if state["iteration_count"] >= 10:
        return "end" # Hard limit
    if last_message.tool_calls:
        return "act" # More tool calls needed
    return "end" # Final answer reached

# Build graph
workflow = StateGraph(AgentState)
workflow.add_node("reason", reason)
workflow.add_node("act", act)

workflow.set_entry_point("reason")
workflow.add_conditional_edges("reason", should_continue, {
    "act": "act",
    "end": END
})
workflow.add_edge("act", "reason") # Cycle: act → reason → act → ...

app = workflow.compile(checkpointer=postgres_checkpointer)

```

Checkpointing — Durability Pattern

Every node execution → state persisted to checkpointer

Benefits:

1. Crash recovery: Agent resumes from last checkpoint on restart
2. Time-travel debugging: Replay execution from any past state
3. Thread forking: Branch from a checkpoint to test alternatives
4. Human interruption: Pause at any node, resume after review

Checkpointer backends:

- MemorySaver → In-process dict (development only)
- SqliteSaver → Local SQLite (single-machine)
- PostgresSaver → Distributed (production)
- RedisSaver → Low-latency, short-lived workflows

Human-in-the-Loop Pattern

```
# Interrupt before executing a destructive action
workflow.add_node("confirm_deletion", confirm_deletion_node)

# Add interrupt point
app = workflow.compile(
    checkpointer=postgres_checkpointer,
    interrupt_before=["confirm_deletion"] # Pause here
)

# Run until interrupt
result = app.invoke(initial_state, config={"thread_id": "abc"})
# Agent pauses at confirm_deletion node, state saved to checkpoint

# Human reviews state, then resumes
app.invoke(None, config={"thread_id": "abc"}) # None = resume from checkpoint
```

Checkpoint Write Amplification — Production Concern

Scenario: 1,000 concurrent users, each agent makes 10 LLM calls

Checkpoint writes:

1,000 users × 10 turns × 1 write/turn = 10,000 writes/hour

State size growth:

Each checkpoint includes full message history

Turn 1: 2 KB

Turn 10: 20 KB (linear growth with conversation)

Turn 50: 100 KB

Optimizations:

1. Compress state (gzip message history): 60-70% size reduction
2. Batch checkpoints (write every 3 turns): 3× write reduction
3. Archive old threads to cold storage (S3) after 24h inactivity
4. Prune message history in state (keep last 20 turns, summarize older)

11.7.3 LangSmith — Observability

What LangSmith Captures

Every chain/agent run creates a trace:

Run trace:

```
├ inputs: {...}
└ outputs: {...}
```

```

└─ latency: 1.23s
└─ tokens: {input: 450, output: 128, total: 578}
└─ cost: $0.0012
└─ child spans:
    └─ ChatOpenAI (0.8s, 450→128 tokens)
    └─ ToolCall: search_web (0.3s)
    └─ ToolCall: query_db (0.1s)

```

Evaluation in LangSmith

```

from langsmith import Client
from langsmith.evaluation import evaluate

def accuracy_evaluator(run, example):
    # LLM-as-judge evaluation
    predicted = run.outputs["answer"]
    reference = example.outputs["answer"]
    score = judge_llm.invoke(f"Is '{predicted}' equivalent to '{reference}'?")
    return {"score": 1 if "yes" in score.lower() else 0}

# Run evaluation against a dataset
results = evaluate(
    lambda inputs: chain.invoke(inputs),
    data="my-qa-dataset",
    evaluators=[accuracy_evaluator],
    experiment_prefix="v2-chain-test"
)

```

Interview Problems

Problem 1: Customer Support Agent with Human Escalation via LangGraph

Scenario: Build a support agent that escalates to a human if confidence is low or the issue is billing-related. Persist state across sessions.

Solution:

```

class SupportState(TypedDict):
    customer_id: str
    conversation: List[dict]
    intent: str          # classified intent
    confidence: float     # classification confidence
    escalated: bool
    human_agent_id: str | None
    resolution: str | None

```

```

def classify_intent(state):
    result = intent_classifier.invoke(state["conversation"])
    return {
        "intent": result.intent,
        "confidence": result.confidence
    }

def should_escalate(state) -> str:
    if state["confidence"] < 0.7:
        return "escalate"
    if state["intent"] in ["billing_dispute", "account_closure"]:
        return "escalate"
    return "self_service"

def escalate(state):
    agent_id = assign_human_agent(state["intent"])
    notify_human(agent_id, state)
    return {"escalated": True, "human_agent_id": agent_id}

workflow.add_conditional_edges(
    "classify_intent",
    should_escalate,
    {"escalate": "escalate", "self_service": "resolve"}
)

# Persist: always
# Never persist: raw PII in tool outputs (redact before checkpoint)
# What to persist:
#   - intent, confidence, escalation_status (always)
#   - conversation summary (not raw if PII-heavy)
#   - resolution outcome (for analytics)

```

Problem 2: Debugging a Flaky LangGraph Agent

Scenario: Your LangGraph agent occasionally produces wrong answers, but only for certain query patterns. How do you diagnose it?

Solution using LangSmith + time-travel:

Diagnosis workflow:

1. LangSmith trace analysis:
 - Filter runs where output quality score < 0.7
 - Group by: intent, model, context_length
 - Identify pattern: "long context + tool_use" → 60% failure rate
2. Time-travel debugging:
 - # Replay specific failing run from checkpoint
 - failing_thread_id = "abc-123"

```
# Get state at each node
history = app.get_state_history({"thread_id": failing_thread_id})

# Find the node where reasoning diverged
for state in history:
    if "wrong_tool_called" in state.values:
        # Examine exactly what context was in model input
        print(state.values["messages"])
```

3. Root cause found:
Long contexts cause the model to lose track of tool schema constraints
4. Fix:
 - Add context compression node before tool-heavy steps
 - Or: Split long tasks into sub-tasks with shorter contexts
5. Regression test:
Add failing cases to LangSmith evaluation dataset
CI pipeline runs evaluation on every deploy

Problem 3: Scaling LangGraph to 10,000 Concurrent Sessions

Scenario: Your support agent needs to handle 10K concurrent sessions. Each session may be long-running (up to 30 minutes). What breaks and how do you fix it?

Solution:

Bottlenecks and fixes:

1. Checkpoint write contention:

Problem: 10,000 writes/second to PostgreSQL

Fix:

 - Use Redis for active sessions (fast writes)
 - Flush to Postgres on session end or every 5 minutes
 - Partition checkpoint table by session_id hash
2. State size growth:

Problem: 30-minute sessions accumulate 200KB+ per session

Fix:

 - Compress after each turn (gzip): ~60KB/session
 - Prune: Keep last 15 turns, summarize older ones
 - Total in-flight state: $10,000 \times 60\text{KB} = 600\text{MB}$ (manageable)
3. LLM call concurrency:

Problem: 10,000 simultaneous LLM calls exhaust API rate limits

Fix:

 - LLM gateway with per-tenant rate limiting
 - Priority queuing: active conversations > idle resumptions
 - Request coalescing for identical prompts (semantic cache)

4. Session timeout cleanup:

Problem: Abandoned sessions hold memory indefinitely

Fix:

- TTL on Redis sessions: 30 minutes of inactivity → archive to S3
- Background job: move cold sessions from Postgres to S3
- On session resume: restore from S3 if not in Postgres